Pandas toolkit Part 3

Syed Afroz Ali

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
frame = pd.DataFrame({"a": ["Yes", "Yes", "No", "No"], "b": range(4)})
frame.describe()
```

Out[2]:

	b
count	4.000000
mean	1.500000
std	1.290994
min	0.000000
25%	0.750000
50%	1.500000
75%	2.250000
max	3.000000

In [3]:
```
frame.describe(include=["object"])
```

Out[3]:

	a
count	4
unique	2
top	Yes
freq	2

In [6]:
```
frame.describe(include=["number"])
```

Out[6]:

	b
count	4.000000
mean	1.500000
std	1.290994
min	0.000000
25%	0.750000
50%	1.500000
75%	2.250000
max	3.000000

In [7]:
```
s1 = pd.Series(np.random.randn(5))
s1
```

Out[7]:
```
0   -0.121086
1    0.060713
2    1.259896
3   -0.161383
4   -2.168469
dtype: float64
```

In [8]:
```
s1.idxmin(), s1.idxmax()
```

Out[8]:
```
(4, 2)
```

In [9]:
```
df1 = pd.DataFrame(np.random.randn(5, 3), columns=["A", "B", "C"])
df1
```

Out[9]:

	A	B	C
0	-1.029309	1.362440	-0.959433
1	0.862846	-0.221771	-1.559672
2	0.735617	0.847179	-0.020883
3	-1.213478	0.416975	1.226910
4	0.020545	-0.211762	-1.391545

In [10]:
```
df1.idxmin(axis=0)
```

Out[10]:
```
A    3
B    1
C    1
dtype: int64
```

In [11]:
```
df1.idxmax(axis=1)
```

Out[11]:
```
0    B
1    A
2    B
3    C
4    A
dtype: object
```

In [12]:
```
df3 = pd.DataFrame([2, 1, 1, 3, np.nan], columns=["A"], index=list("edcba"))
df3
```

Out[12]:

	A
e	2.0
d	1.0
c	1.0
b	3.0
a	NaN

In [13]:
```
data = np.random.randint(0, 7, size=50)
data
```

Out[13]:
```
array([6, 5, 6, 2, 6, 2, 3, 5, 1, 3, 1, 3, 1, 5, 2, 6, 6, 4, 4, 6, 0, 5,
       0, 3, 5, 4, 1, 2, 2, 6, 1, 6, 1, 0, 4, 4, 0, 4, 3, 5, 6, 0, 6, 4,
       5, 5, 1, 1, 2, 5])
```

In [14]:
```
s = pd.Series(data)
s.value_counts()
```

Out[14]:
```
6    10
5     9
1     8
4     7
2     6
3     5
0     5
dtype: int64
```

In [15]:
```
data = {"a": [1, 2, 3, 4], "b": ["x", "x", "y", "y"]}
frame = pd.DataFrame(data)
frame.value_counts()
```

Out[15]:
```
a  b
1  x    1
2  x    1
3  y    1
4  y    1
dtype: int64
```

In [16]:
```
s5 = pd.Series([1, 1, 3, 3, 3, 5, 5, 7, 7, 7])
s5.mode()
```

Out[16]:
```
0    3
1    7
dtype: int64
```

In [17]:
```
df5 = pd.DataFrame({
"A": np.random.randint(0, 7, size=50),
"B": np.random.randint(-10, 15, size=50),
})
df5.mode()
```

Out[17]:

	A	B
0	0.0	-9
1	NaN	-3

In [18]:
```
arr = np.random.randn(20)
factor = pd.cut(arr, 4)
factor
```

Out[18]:
```
[(-0.245, 0.809], (0.809, 1.863], (-1.303, -0.245], (-0.245, 0.809], (-0.245,
0.809], ..., (-0.245, 0.809], (-0.245, 0.809], (1.863, 2.917], (-0.245, 0.80
9], (-0.245, 0.809]]
Length: 20
Categories (4, interval[float64, right]): [(-1.303, -0.245] < (-0.245, 0.809]
< (0.809, 1.863] < (1.863, 2.917]]
```

In [19]:
```
factor = pd.cut(arr, [-5, -1, 0, 1, 5])
factor
```

Out[19]:
```
[(0, 1], (1, 5], (-1, 0], (-1, 0], (0, 1], ..., (0, 1], (0, 1], (1, 5], (-1,
0], (-1, 0]]
Length: 20
Categories (4, interval[int64, right]): [(-5, -1] < (-1, 0] < (0, 1] < (1,
5]]
```

In [20]:
```
arr = np.random.randn(30)
factor = pd.qcut(arr, [0, 0.25, 0.5, 0.75, 1])
factor
```

Out[20]:
```
[(-0.204, 0.428], (-0.75, -0.204], (-3.0669999999999997, -0.75], (-0.75, -0.2
04], (-0.75, -0.204], ..., (-3.0669999999999997, -0.75], (-0.204, 0.428], (-
3.0669999999999997, -0.75], (-0.204, 0.428], (-0.75, -0.204]]
Length: 30
Categories (4, interval[float64, right]): [(-3.0669999999999997, -0.75] < (-
0.75, -0.204] < (-0.204, 0.428] < (0.428, 2.156]]
```

In [21]:
```
arr = np.random.randn(20)
factor = pd.cut(arr, [-np.inf, 0, np.inf])
factor
```

Out[21]:
```
[(-inf, 0.0], (-inf, 0.0], (0.0, inf], (0.0, inf], (0.0, inf], ..., (-inf, 0.
0], (0.0, inf], (0.0, inf], (-inf, 0.0], (-inf, 0.0]]
Length: 20
Categories (2, interval[float64, right]): [(-inf, 0.0] < (0.0, inf]]
```

In [23]:
```
def extract_city_name(df):
    df["city_name"] = df["city_and_code"].str.split(",").str.get(0)
    return df
def add_country_name(df, country_name=None):
    col = "city_name"
    df["city_and_country"] = df[col] + country_name
    return df
df_p = pd.DataFrame({"city_and_code": ["Chicago, IL"]})


add_country_name(extract_city_name(df_p), country_name="US")
```

Out[23]:

	city_and_code	city_name	city_and_country
0	Chicago, IL	Chicago	ChicagoUS

In [24]:
```
df_p.pipe(extract_city_name).pipe(add_country_name, country_name="US")
```

Out[24]:

	city_and_code	city_name	city_and_country
0	Chicago, IL	Chicago	ChicagoUS

In [25]:
```
import statsmodels.formula.api as sm
bb = pd.read_csv("baseball.csv", index_col="id")
(
bb.query("h > 0")
.assign(ln_h=lambda df: np.log(df.h))
.pipe((sm.ols, "data"), "hr ~ ln_h + year + g + C(lg)")
.fit()
.summary()
)
```

Out[25]:

OLS Regression Results

Dep. Variable:	hr	**R-squared:**	0.685
Model:	OLS	**Adj. R-squared:**	0.665
Method:	Least Squares	**F-statistic:**	34.28
Date:	Thu, 22 Sep 2022	**Prob (F-statistic):**	3.48e-15
Time:	18:53:34	**Log-Likelihood:**	-205.92
No. Observations:	68	**AIC:**	421.8
Df Residuals:	63	**BIC:**	432.9
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-8484.7720	4664.146	-1.819	0.074	-1.78e+04	835.780
C(lg)[T.NL]	-2.2736	1.325	-1.716	0.091	-4.922	0.375
ln_h	-1.3542	0.875	-1.547	0.127	-3.103	0.395
year	4.2277	2.324	1.819	0.074	-0.417	8.872
g	0.1841	0.029	6.258	0.000	0.125	0.243

Omnibus:	10.875	**Durbin-Watson:**	1.999
Prob(Omnibus):	0.004	**Jarque-Bera (JB):**	17.298
Skew:	0.537	**Prob(JB):**	0.000175
Kurtosis:	5.225	**Cond. No.**	1.49e+07

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.49e+07. This might indicate that there are strong multicollinearity or other numerical problems.

In [27]:
```
df5.apply(np.mean)
```

Out[27]:
```
A    2.96
B    2.06
dtype: float64
```

In [28]:
```
df5.apply(np.mean, axis=1)
```

Out[28]:
```
0     3.0
1     4.5
2     6.0
3     1.0
4     3.0
5     6.5
6    -2.5
7    -4.5
8     5.0
9     8.5
10    2.0
11    0.0
12    3.0
13    2.0
14   -1.5
15    5.5
16    6.5
17   -3.5
18    2.5
19    0.5
20    1.0
21    6.5
22   -1.5
23    3.0
24   -3.5
25    7.5
26    5.0
27   -2.5
28    2.0
29    3.0
30    8.5
31    4.5
32    0.0
33    9.0
34    6.5
35    8.0
36   -2.5
37    0.0
38    1.5
39    0.5
40   -4.0
41    0.5
42    7.5
43   -0.5
44    0.0
45    7.0
46    4.0
47    5.0
48    1.0
49    1.0
dtype: float64
```

In [29]:
```
df5.apply(lambda x: x.max() - x.min())
```

Out[29]:
```
A     6
B    23
dtype: int64
```

In [30]:
```
df5.apply(np.cumsum)
```

Out[30]:

	A	B
0	0	6
1	2	13
2	2	25
3	7	22
4	7	28
5	12	36
6	15	28
7	15	19
8	17	27
9	21	40
10	22	43
11	22	43
12	25	46
13	31	44
14	37	35
15	39	44
16	41	55
17	44	45
18	50	44
19	51	44
20	51	46
21	56	54
22	61	46
23	62	51
24	64	42
25	70	51
26	73	58
27	78	48
28	83	47
29	87	49
30	93	60
31	93	69
32	97	65
33	103	77
34	104	89
35	110	99

	A	B
36	111	93
37	114	90
38	119	88
39	123	85
40	124	76
41	128	73
42	132	84
43	138	77
44	143	72
45	144	85
46	146	91
47	146	101
48	146	103
49	148	103

In [32]:
```
df5.apply(np.exp)
```

Out[32]:

	A	B
0	1.000000	403.428793
1	7.389056	1096.633158
2	1.000000	162754.791419
3	148.413159	0.049787
4	1.000000	403.428793
5	148.413159	2980.957987
6	20.085537	0.000335
7	1.000000	0.000123
8	7.389056	2980.957987
9	54.598150	442413.392009
10	2.718282	20.085537
11	1.000000	1.000000
12	20.085537	20.085537
13	403.428793	0.135335
14	403.428793	0.000123
15	7.389056	8103.083928
16	7.389056	59874.141715
17	20.085537	0.000045
18	403.428793	0.367879
19	2.718282	1.000000
20	1.000000	7.389056
21	148.413159	2980.957987
22	148.413159	0.000335
23	2.718282	148.413159
24	7.389056	0.000123
25	403.428793	8103.083928
26	20.085537	1096.633158
27	148.413159	0.000045
28	148.413159	0.367879
29	54.598150	7.389056
30	403.428793	59874.141715
31	1.000000	8103.083928
32	54.598150	0.018316
33	403.428793	162754.791419
34	2.718282	162754.791419
35	403.428793	22026.465795

	A	B
36	2.718282	0.002479
37	20.085537	0.049787
38	148.413159	0.135335
39	54.598150	0.049787
40	2.718282	0.000123
41	54.598150	0.049787
42	54.598150	59874.141715
43	403.428793	0.000912
44	148.413159	0.006738
45	2.718282	442413.392009
46	7.389056	403.428793
47	1.000000	22026.465795
48	1.000000	7.389056
49	7.389056	1.000000

In [33]:
```
df5.apply("mean")
```

Out[33]:
```
A    2.96
B    2.06
dtype: float64
```

In [35]:
```
df5.apply("mean", axis=1)
```

Out[35]:
```
0     3.0
1     4.5
2     6.0
3     1.0
4     3.0
5     6.5
6    -2.5
7    -4.5
8     5.0
9     8.5
10    2.0
11    0.0
12    3.0
13    2.0
14   -1.5
15    5.5
16    6.5
17   -3.5
18    2.5
19    0.5
20    1.0
21    6.5
22   -1.5
23    3.0
24   -3.5
25    7.5
26    5.0
27   -2.5
28    2.0
29    3.0
30    8.5
31    4.5
32    0.0
33    9.0
34    6.5
35    8.0
36   -2.5
37    0.0
38    1.5
39    0.5
40   -4.0
41    0.5
42    7.5
43   -0.5
44    0.0
45    7.0
46    4.0
47    5.0
48    1.0
49    1.0
dtype: float64
```

In [36]:
```
def subtract_and_divide(x, sub, divide=1):
    return (x - sub) / divide

df5.apply(subtract_and_divide, args=(5,), divide=3)
```

Out[36]:

	A	B
0	-1.666667	0.333333
1	-1.000000	0.666667
2	-1.666667	2.333333
3	0.000000	-2.666667
4	-1.666667	0.333333
5	0.000000	1.000000
6	-0.666667	-4.333333
7	-1.666667	-4.666667
8	-1.000000	1.000000
9	-0.333333	2.666667
10	-1.333333	-0.666667
11	-1.666667	-1.666667
12	-0.666667	-0.666667
13	0.333333	-2.333333
14	0.333333	-4.666667
15	-1.000000	1.333333
16	-1.000000	2.000000
17	-0.666667	-5.000000
18	0.333333	-2.000000
19	-1.333333	-1.666667
20	-1.666667	-1.000000
21	0.000000	1.000000
22	0.000000	-4.333333
23	-1.333333	0.000000
24	-1.000000	-4.666667
25	0.333333	1.333333
26	-0.666667	0.666667
27	0.000000	-5.000000
28	0.000000	-2.000000
29	-0.333333	-1.000000
30	0.333333	2.000000
31	-1.666667	1.333333
32	-0.333333	-3.000000
33	0.333333	2.333333
34	-1.333333	2.333333
35	0.333333	1.666667

	A	B
36	-1.333333	-3.666667
37	-0.666667	-2.666667
38	0.000000	-2.333333
39	-0.333333	-2.666667
40	-1.333333	-4.666667
41	-0.333333	-2.666667
42	-0.333333	2.000000
43	0.333333	-4.000000
44	0.000000	-3.333333
45	-1.333333	2.666667
46	-1.000000	0.333333
47	-1.666667	1.666667
48	-1.666667	-1.000000
49	-1.000000	-1.666667

In [37]:
```
tsdf = pd.DataFrame(
np.random.randn(10, 3),
columns=["A", "B", "C"],
index=pd.date_range("1/1/2000", periods=10),
)

tsdf.iloc[3:7] = np.nan
tsdf
```

Out[37]:

	A	B	C
2000-01-01	1.100146	-0.594632	-0.486077
2000-01-02	-1.281338	-0.032859	0.675010
2000-01-03	-1.250284	1.207627	-0.363746
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	1.381345	-1.236094	2.241808
2000-01-09	0.209783	0.166198	-0.248163
2000-01-10	2.156381	0.918886	-2.077679

In [38]:
```
tsdf.agg(np.sum)
```

Out[38]:
```
A    2.316033
B    0.429125
C   -0.258846
dtype: float64
```

In [39]:
```
tsdf.agg("sum")
```

Out[39]:
```
A    2.316033
B    0.429125
C   -0.258846
dtype: float64
```

In [40]:
```
tsdf.sum()
```

Out[40]:
```
A    2.316033
B    0.429125
C   -0.258846
dtype: float64
```

In [41]:
```
tsdf["A"].agg("sum")
```

Out[41]:
```
2.3160328735804745
```

In [42]:
```
tsdf.agg(["sum", "mean"])
```

Out[42]:

	A	B	C
sum	2.316033	0.429125	-0.258846
mean	0.386005	0.071521	-0.043141

In [43]:
```
tsdf["A"].agg(["sum", "mean"])
```

Out[43]:
```
sum     2.316033
mean    0.386005
Name: A, dtype: float64
```

In [44]:
```
tsdf["A"].agg(["sum", lambda x: x.mean()])
```

Out[44]:
```
sum         2.316033
<lambda>    0.386005
Name: A, dtype: float64
```

In [45]:
```
def mymean(x):
    return x.mean()

tsdf["A"].agg(["sum", mymean])
```

Out[45]:
```
sum       2.316033
mymean    0.386005
Name: A, dtype: float64
```

In [46]:
```
tsdf.agg({"A": "mean", "B": "sum"})
```

Out[46]:
```
A    0.386005
B    0.429125
dtype: float64
```

In [47]:
```
tsdf.agg({"A": ["mean", "min"], "B": "sum"})
```

Out[47]:

	A	B
mean	0.386005	NaN
min	-1.281338	NaN
sum	NaN	0.429125

In [48]:
```
mdf = pd.DataFrame({
"A": [1, 2, 3],
"B": [1.0, 2.0, 3.0],
"C": ["foo", "bar", "baz"],
"D": pd.date_range("20130101", periods=3),
})
```

In [49]:
```
mdf.agg(["min", "sum"])
```

Out[49]:

	A	B	C	D
min	1	1.0	bar	2013-01-01
sum	6	6.0	foobarbaz	NaT

In [50]:
```
# Custom describe

from functools import partial
q_25 = partial(pd.Series.quantile, q=0.25)
q_25.__name__ = "25%"
q_75 = partial(pd.Series.quantile, q=0.75)
q_75.__name__ = "75%"
tsdf.agg(["count", "mean", "std", "min", q_25, "median", q_75, "max"])
```

Out[50]:

	A	B	C
count	6.000000	6.000000	6.000000
mean	0.386005	0.071521	-0.043141
std	1.422916	0.914575	1.429482
min	-1.281338	-1.236094	-2.077679
25%	-0.885267	-0.454189	-0.455494
median	0.654965	0.066669	-0.305954
75%	1.311045	0.730714	0.444217
max	2.156381	1.207627	2.241808

In [52]:
```
tsdf = pd.DataFrame(
np.random.randn(10, 3),
columns=["A", "B", "C"],
index=pd.date_range("1/1/2000", periods=10),
)

tsdf.iloc[3:7] = np.nan
tsdf
```

Out[52]:

	A	B	C
2000-01-01	-0.632673	0.474561	-0.798479
2000-01-02	1.250986	-0.578337	1.065323
2000-01-03	-0.998635	-1.218509	-0.738105
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	-0.683195	1.623150	0.090563
2000-01-09	-0.118824	-0.426729	-1.098490
2000-01-10	-1.150192	0.214560	1.337532

In [53]:
```
tsdf.transform(np.abs)
```

Out[53]:

	A	B	C
2000-01-01	0.632673	0.474561	0.798479
2000-01-02	1.250986	0.578337	1.065323
2000-01-03	0.998635	1.218509	0.738105
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.683195	1.623150	0.090563
2000-01-09	0.118824	0.426729	1.098490
2000-01-10	1.150192	0.214560	1.337532

In [54]:
```
tsdf.transform("abs")
```

Out[54]:

	A	B	C
2000-01-01	0.632673	0.474561	0.798479
2000-01-02	1.250986	0.578337	1.065323
2000-01-03	0.998635	1.218509	0.738105
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.683195	1.623150	0.090563
2000-01-09	0.118824	0.426729	1.098490
2000-01-10	1.150192	0.214560	1.337532

In [55]:
```
tsdf.transform(lambda x: x.abs())
```

Out[55]:

	A	B	C
2000-01-01	0.632673	0.474561	0.798479
2000-01-02	1.250986	0.578337	1.065323
2000-01-03	0.998635	1.218509	0.738105
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.683195	1.623150	0.090563
2000-01-09	0.118824	0.426729	1.098490
2000-01-10	1.150192	0.214560	1.337532

In [56]:
```
np.abs(tsdf)
```

Out[56]:

	A	B	C
2000-01-01	0.632673	0.474561	0.798479
2000-01-02	1.250986	0.578337	1.065323
2000-01-03	0.998635	1.218509	0.738105
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.683195	1.623150	0.090563
2000-01-09	0.118824	0.426729	1.098490
2000-01-10	1.150192	0.214560	1.337532

In [57]:
```
tsdf["A"].transform(np.abs)
```

Out[57]:
```
2000-01-01    0.632673
2000-01-02    1.250986
2000-01-03    0.998635
2000-01-04         NaN
2000-01-05         NaN
2000-01-06         NaN
2000-01-07         NaN
2000-01-08    0.683195
2000-01-09    0.118824
2000-01-10    1.150192
Freq: D, Name: A, dtype: float64
```

In [58]:
```
tsdf.transform([np.abs, lambda x: x + 1])
```

Out[58]:

	A		B		C	
	absolute	**<lambda>**	**absolute**	**<lambda>**	**absolute**	**<lambda>**
2000-01-01	0.632673	0.367327	0.474561	1.474561	0.798479	0.201521
2000-01-02	1.250986	2.250986	0.578337	0.421663	1.065323	2.065323
2000-01-03	0.998635	0.001365	1.218509	-0.218509	0.738105	0.261895
2000-01-04	NaN	NaN	NaN	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN	NaN	NaN	NaN
2000-01-08	0.683195	0.316805	1.623150	2.623150	0.090563	1.090563
2000-01-09	0.118824	0.881176	0.426729	0.573271	1.098490	-0.098490
2000-01-10	1.150192	-0.150192	0.214560	1.214560	1.337532	2.337532

In [59]:
```
tsdf["A"].transform([np.abs, lambda x: x + 1])
```

Out[59]:

	absolute	<lambda>
2000-01-01	0.632673	0.367327
2000-01-02	1.250986	2.250986
2000-01-03	0.998635	0.001365
2000-01-04	NaN	NaN
2000-01-05	NaN	NaN
2000-01-06	NaN	NaN
2000-01-07	NaN	NaN
2000-01-08	0.683195	0.316805
2000-01-09	0.118824	0.881176
2000-01-10	1.150192	-0.150192

In [60]:
```
tsdf.transform({"A": np.abs, "B": lambda x: x + 1})
```

Out[60]:

	A	B
2000-01-01	0.632673	1.474561
2000-01-02	1.250986	0.421663
2000-01-03	0.998635	-0.218509
2000-01-04	NaN	NaN
2000-01-05	NaN	NaN
2000-01-06	NaN	NaN
2000-01-07	NaN	NaN
2000-01-08	0.683195	2.623150
2000-01-09	0.118824	0.573271
2000-01-10	1.150192	1.214560

In [68]:
```
df = pd.DataFrame({
"one": pd.Series(np.random.randn(3), index=["a", "b", "c"]),
"two": pd.Series(np.random.randn(4), index=["a", "b", "c", "d"]),
"three": pd.Series(np.random.randn(3), index=["b", "c", "d"]),
})

df
```

Out[68]:

	one	two	three
a	-0.789990	-0.020558	NaN
b	0.572949	-0.778513	-0.450913
c	-0.367262	1.963174	-0.203882
d	NaN	-0.313509	0.001874

In [69]:
```
df["three"] = df["one"] * df["two"]
df["flag"] = df["one"] > 2
df
```

Out[69]:

	one	two	three	flag
a	-0.789990	-0.020558	0.016241	False
b	0.572949	-0.778513	-0.446048	False
c	-0.367262	1.963174	-0.721000	False
d	NaN	-0.313509	NaN	False

In [70]:
```
def f(x):
    return len(str(x))
df["one"].map(f)
```

Out[70]:
```
a    19
b    18
c    19
d     3
Name: one, dtype: int64
```

In [71]:
```
df.applymap(f)
```

Out[71]:

	one	two	three	flag
a	19	21	19	5
b	18	19	19	5
c	19	18	19	5
d	3	19	3	5

In [72]:
```
s = pd.Series(
["six", "seven", "six", "seven", "six"], index=["a", "b", "c", "d", "e"]
)

t = pd.Series({"six": 6.0, "seven": 7.0})
s
```

Out[72]:
```
a      six
b    seven
c      six
d    seven
e      six
dtype: object
```

In [73]:
```
s.map(t)
```

Out[73]:
```
a    6.0
b    7.0
c    6.0
d    7.0
e    6.0
dtype: float64
```

In [74]:
```
s = pd.Series(np.random.randn(5), index=["a", "b", "c", "d", "e"])
s
```

Out[74]:
```
a   -0.727970
b    1.102445
c    1.113834
d   -0.195407
e    0.238101
dtype: float64
```

In [75]:
```
s.reindex(["e", "b", "f", "d"])
```

Out[75]:
```
e    0.238101
b    1.102445
f         NaN
d   -0.195407
dtype: float64
```

In [76]:
```
df.reindex(index=["c", "f", "b"], columns=["three", "two", "one"])
```

Out[76]:

	three	two	one
c	-0.721000	1.963174	-0.367262
f	NaN	NaN	NaN
b	-0.446048	-0.778513	0.572949

In [77]:
```
df.reindex(["c", "f", "b"], axis="index")
```

Out[77]:

	one	two	three	flag
c	-0.367262	1.963174	-0.721000	False
f	NaN	NaN	NaN	NaN
b	0.572949	-0.778513	-0.446048	False

In [78]:
```
rs = s.reindex(df.index)
rs
```

Out[78]:
```
a   -0.727970
b    1.102445
c    1.113834
d   -0.195407
dtype: float64
```

In [79]:
```
df.reindex(["c", "f", "b"], axis="index")
```

Out[79]:

	one	two	three	flag
c	-0.367262	1.963174	-0.721000	False
f	NaN	NaN	NaN	NaN
b	0.572949	-0.778513	-0.446048	False

In [80]:
```
df.reindex(["three", "two", "one"], axis="columns")
```

Out[80]:

	three	two	one
a	0.016241	-0.020558	-0.789990
b	-0.446048	-0.778513	0.572949
c	-0.721000	1.963174	-0.367262
d	NaN	-0.313509	NaN

In [82]:
```
df.reindex_like(df)
```

Out[82]:

	one	two	three	flag
a	-0.789990	-0.020558	0.016241	False
b	0.572949	-0.778513	-0.446048	False
c	-0.367262	1.963174	-0.721000	False
d	NaN	-0.313509	NaN	False

In [83]:
```
s = pd.Series(np.random.randn(5), index=["a", "b", "c", "d", "e"])
s1 = s[:4]
s2 = s[1:]
s1.align(s2)
```

Out[83]:
```
(a   -0.511169
 b   -0.185304
 c    0.725502
 d   -1.852033
 e         NaN
 dtype: float64,
 a         NaN
 b   -0.185304
 c    0.725502
 d   -1.852033
 e   -1.251512
 dtype: float64)
```

In [84]:
```
s1.align(s2, join="inner")
```

Out[84]:
```
(b   -0.185304
 c    0.725502
 d   -1.852033
 dtype: float64,
 b   -0.185304
 c    0.725502
 d   -1.852033
 dtype: float64)
```

In [85]:
```
s1.align(s2, join="left")
```

Out[85]:
```
(a   -0.511169
 b   -0.185304
 c    0.725502
 d   -1.852033
 dtype: float64,
 a         NaN
 b   -0.185304
 c    0.725502
 d   -1.852033
 dtype: float64)
```

In [88]:
```
df.align(df5, join="inner")
```

Out[88]:
```
(Empty DataFrame
 Columns: []
 Index: [],
 Empty DataFrame
 Columns: []
 Index: [])
```

In [89]:
```
df.align(df5, join="inner", axis=0)
```

Out[89]:
```
(Empty DataFrame
 Columns: [one, two, three, flag]
 Index: [],
 Empty DataFrame
 Columns: [A, B]
 Index: [])
```

In [91]:
```
df.align(df5.iloc[0], axis=1)
```

Out[91]:
```
(    A   B   flag       one     three       two
 a NaN NaN  False -0.789990  0.016241 -0.020558
 b NaN NaN  False  0.572949 -0.446048 -0.778513
 c NaN NaN  False -0.367262 -0.721000  1.963174
 d NaN NaN  False       NaN       NaN -0.313509,
 A         0.0
 B         6.0
 flag      NaN
 one       NaN
 three     NaN
 two       NaN
 Name: 0, dtype: float64)
```

In [92]:
```
rng = pd.date_range("1/3/2000", periods=8)
ts = pd.Series(np.random.randn(8), index=rng)
ts2 = ts[[0, 3, 6]]
ts
```

Out[92]:
```
2000-01-03   -1.310621
2000-01-04   -0.992201
2000-01-05   -1.394069
2000-01-06    0.820258
2000-01-07   -1.331111
2000-01-08    0.116894
2000-01-09   -0.452949
2000-01-10    1.596265
Freq: D, dtype: float64
```

In [93]:
```
ts2.reindex(ts.index)
```

Out[93]:
```
2000-01-03   -1.310621
2000-01-04         NaN
2000-01-05         NaN
2000-01-06    0.820258
2000-01-07         NaN
2000-01-08         NaN
2000-01-09   -0.452949
2000-01-10         NaN
Freq: D, dtype: float64
```

In [94]:
```
ts2.reindex(ts.index, method="ffill")
```

Out[94]:
```
2000-01-03   -1.310621
2000-01-04   -1.310621
2000-01-05   -1.310621
2000-01-06    0.820258
2000-01-07    0.820258
2000-01-08    0.820258
2000-01-09   -0.452949
2000-01-10   -0.452949
Freq: D, dtype: float64
```

In [95]:
```
ts2.reindex(ts.index, method="bfill")
```

Out[95]:
```
2000-01-03   -1.310621
2000-01-04    0.820258
2000-01-05    0.820258
2000-01-06    0.820258
2000-01-07   -0.452949
2000-01-08   -0.452949
2000-01-09   -0.452949
2000-01-10         NaN
Freq: D, dtype: float64
```

In [96]:
```
ts2.reindex(ts.index, method="nearest")
```

Out[96]:
```
2000-01-03   -1.310621
2000-01-04   -1.310621
2000-01-05    0.820258
2000-01-06    0.820258
2000-01-07    0.820258
2000-01-08   -0.452949
2000-01-09   -0.452949
2000-01-10   -0.452949
Freq: D, dtype: float64
```

In [97]:
```
ts2.reindex(ts.index).fillna(method="ffill")
```

Out[97]:
```
2000-01-03   -1.310621
2000-01-04   -1.310621
2000-01-05   -1.310621
2000-01-06    0.820258
2000-01-07    0.820258
2000-01-08    0.820258
2000-01-09   -0.452949
2000-01-10   -0.452949
Freq: D, dtype: float64
```

In [98]:
```
ts2.reindex(ts.index, method="ffill", limit=1)
```

Out[98]:
```
2000-01-03   -1.310621
2000-01-04   -1.310621
2000-01-05         NaN
2000-01-06    0.820258
2000-01-07    0.820258
2000-01-08         NaN
2000-01-09   -0.452949
2000-01-10   -0.452949
Freq: D, dtype: float64
```

In [99]:
```
ts2.reindex(ts.index, method="ffill", tolerance="1 day")
```

Out[99]:
```
2000-01-03   -1.310621
2000-01-04   -1.310621
2000-01-05         NaN
2000-01-06    0.820258
2000-01-07    0.820258
2000-01-08         NaN
2000-01-09   -0.452949
2000-01-10   -0.452949
Freq: D, dtype: float64
```

In [100]:
```
df.drop(["a", "d"], axis=0)
```

Out[100]:

	one	two	three	flag
b	0.572949	-0.778513	-0.446048	False
c	-0.367262	1.963174	-0.721000	False

In [101]:
```
df.drop(["one"], axis=1)
```

Out[101]:

	two	three	flag
a	-0.020558	0.016241	False
b	-0.778513	-0.446048	False
c	1.963174	-0.721000	False
d	-0.313509	NaN	False

In [102]:
```
df.reindex(df.index.difference(["a", "d"]))
```

Out[102]:

	one	two	three	flag
b	0.572949	-0.778513	-0.446048	False
c	-0.367262	1.963174	-0.721000	False

In [103]:
```
s.rename(str.upper)
```

Out[103]:
```
A   -0.511169
B   -0.185304
C    0.725502
D   -1.852033
E   -1.251512
dtype: float64
```

In [104]:
```
df.rename(
columns={"one": "foo", "two": "bar"},
index={"a": "apple", "b": "banana", "d": "durian"})
```

Out[104]:

	foo	bar	three	flag
apple	-0.789990	-0.020558	0.016241	False
banana	0.572949	-0.778513	-0.446048	False
c	-0.367262	1.963174	-0.721000	False
durian	NaN	-0.313509	NaN	False

In [105]:
```
df.rename({"one": "foo", "two": "bar"}, axis="columns")
```

Out[105]:

	foo	bar	three	flag
a	-0.789990	-0.020558	0.016241	False
b	0.572949	-0.778513	-0.446048	False
c	-0.367262	1.963174	-0.721000	False
d	NaN	-0.313509	NaN	False

In [106]:
```
df.rename({"a": "apple", "b": "banana", "d": "durian"}, axis="index")
```

Out[106]:

	one	two	three	flag
apple	-0.789990	-0.020558	0.016241	False
banana	0.572949	-0.778513	-0.446048	False
c	-0.367262	1.963174	-0.721000	False
durian	NaN	-0.313509	NaN	False

In [107]:
```
s.rename("scalar-name")
```

Out[107]:
```
a   -0.511169
b   -0.185304
c    0.725502
d   -1.852033
e   -1.251512
Name: scalar-name, dtype: float64
```

In [108]:
```
df = pd.DataFrame(
{"x": [1, 2, 3, 4, 5, 6], "y": [10, 20, 30, 40, 50, 60]},
index=pd.MultiIndex.from_product(
[["a", "b", "c"], [1, 2]], names=["let", "num"]
))

df
```

Out[108]:

		x	y
let	num		
a	1	1	10
	2	2	20
b	1	3	30
	2	4	40
c	1	5	50
	2	6	60

In [109]:
```
df.rename_axis(index={"let": "abc"})
```

Out[109]:

		x	y
abc	num		
a	1	1	10
	2	2	20
b	1	3	30
	2	4	40
c	1	5	50
	2	6	60

In [110]:
```
df.rename_axis(index=str.upper)
```

Out[110]:

		x	y
LET	NUM		
a	1	1	10
	2	2	20
b	1	3	30
	2	4	40
c	1	5	50
	2	6	60

In [111]:
```
df = pd.DataFrame({"col1": np.random.randn(3), "col2": np.random.randn(3)}, inc

for col in df:
    print(col)
```

```
col1
col2
```

In [112]:
```
df = pd.DataFrame({"a": [1, 2, 3], "b": ["a", "b", "c"]})
In [257]: for index, row in df.iterrows():
        row["a"] = 10
df
```

Out[112]:

	a	b
0	1	a
1	2	b
2	3	c

In [113]:
```
for label, ser in df.items():
    print(label)
    print(ser)
```

```
a
0    1
1    2
2    3
Name: a, dtype: int64
b
0    a
1    b
2    c
Name: b, dtype: object
```

In [114]:
```
for row_index, row in df.iterrows():
    print(row_index, row, sep="\n")
```

```
0
a    1
b    a
Name: 0, dtype: object
1
a    2
b    b
Name: 1, dtype: object
2
a    3
b    c
Name: 2, dtype: object
```

In [115]:
```
df_orig = pd.DataFrame([[1, 1.5]], columns=["int", "float"])
df_orig.dtypes
```

Out[115]:
```
int        int64
float    float64
dtype: object
```

In [116]:
```
row = next(df_orig.iterrows())[1]
row
```

Out[116]:
```
int      1.0
float    1.5
Name: 0, dtype: float64
```

In [117]:
```
row["int"].dtype
```

Out[117]:
```
dtype('float64')
```

In [118]:
```
df_orig["int"].dtype
```

Out[118]:
```
dtype('int64')
```

In [119]:
```
df2 = pd.DataFrame({"x": [1, 2, 3], "y": [4, 5, 6]})
print(df2)
```

```
   x  y
0  1  4
1  2  5
2  3  6
```

In [120]:
```
df2_t = pd.DataFrame({idx: values for idx, values in df2.iterrows()})
print(df2_t)
```

```
   0  1  2
x  1  2  3
y  4  5  6
```

In [121]:
```
for row in df.itertuples():
    print(row)
```

```
Pandas(Index=0, a=1, b='a')
Pandas(Index=1, a=2, b='b')
Pandas(Index=2, a=3, b='c')
```

In [122]:
```
s = pd.Series(pd.date_range("20130101 09:10:12", periods=4))
s
```

Out[122]:
```
0   2013-01-01 09:10:12
1   2013-01-02 09:10:12
2   2013-01-03 09:10:12
3   2013-01-04 09:10:12
dtype: datetime64[ns]
```

In [123]:
```
s.dt.hour
```

Out[123]:
```
0    9
1    9
2    9
3    9
dtype: int64
```

In [124]:
```
s.dt.second
```

Out[124]:
```
0    12
1    12
2    12
3    12
dtype: int64
```

In [125]:
```
s.dt.day
```

Out[125]:
```
0    1
1    2
2    3
3    4
dtype: int64
```

In [126]:
```
s[s.dt.day == 2]
```

Out[126]:
```
1   2013-01-02 09:10:12
dtype: datetime64[ns]
```

In [127]:
```
stz = s.dt.tz_localize("US/Eastern")
stz
```

Out[127]:
```
0   2013-01-01 09:10:12-05:00
1   2013-01-02 09:10:12-05:00
2   2013-01-03 09:10:12-05:00
3   2013-01-04 09:10:12-05:00
dtype: datetime64[ns, US/Eastern]
```

In [128]:
```
s.dt.tz_localize("UTC").dt.tz_convert("US/Eastern")
```

Out[128]:
```
0   2013-01-01 04:10:12-05:00
1   2013-01-02 04:10:12-05:00
2   2013-01-03 04:10:12-05:00
3   2013-01-04 04:10:12-05:00
dtype: datetime64[ns, US/Eastern]
```

In [129]:
```
s = pd.Series(pd.date_range("20130101", periods=4))
s
```

Out[129]:
```
0   2013-01-01
1   2013-01-02
2   2013-01-03
3   2013-01-04
dtype: datetime64[ns]
```

In [130]:
```
s.dt.strftime("%Y/%m/%d")
```

Out[130]:
```
0    2013/01/01
1    2013/01/02
2    2013/01/03
3    2013/01/04
dtype: object
```

In [132]:
```
s = pd.Series(pd.period_range("20130101", periods=4))
s
```

Out[132]:
```
0    2013-01-01
1    2013-01-02
2    2013-01-03
3    2013-01-04
dtype: period[D]
```

In [133]:
```
s.dt.strftime("%Y/%m/%d")
```

Out[133]:
```
0    2013/01/01
1    2013/01/02
2    2013/01/03
3    2013/01/04
dtype: object
```

In [134]:
```
s = pd.Series(pd.period_range("20130101", periods=4, freq="D"))
s
```

Out[134]:
```
0    2013-01-01
1    2013-01-02
2    2013-01-03
3    2013-01-04
dtype: period[D]
```

In [135]:
```
s.dt.year
```

Out[135]:
```
0    2013
1    2013
2    2013
3    2013
dtype: int64
```

In [136]:
```
s.dt.day
```

Out[136]:
```
0    1
1    2
2    3
3    4
dtype: int64
```

In [137]:
```
s = pd.Series(pd.timedelta_range("1 day 00:00:05", periods=4, freq="s"))
s
```

Out[137]:
```
0   1 days 00:00:05
1   1 days 00:00:06
2   1 days 00:00:07
3   1 days 00:00:08
dtype: timedelta64[ns]
```

In [138]:
```
s.dt.days
```

Out[138]:
```
0    1
1    1
2    1
3    1
dtype: int64
```

In [139]:
```
s.dt.seconds
```

Out[139]:
```
0    5
1    6
2    7
3    8
dtype: int64
```

In [140]:
```
s.dt.components
```

Out[140]:

	days	hours	minutes	seconds	milliseconds	microseconds	nanoseconds
0	1	0	0	5	0	0	0
1	1	0	0	6	0	0	0
2	1	0	0	7	0	0	0
3	1	0	0	8	0	0	0

Syed Afroz Ali

In [ ]:
```

```